

PROPOSAL TUGAS AKHIR

**IMPLEMENTASI NASI CUSTOM (NESTED ARGOCD SERVICE
INTEGRATION – CASCADING UNIFIED SERVICE TEMPLATE
ORCHESTRATION & MANIFEST) UNTUK MANAJEMEN DEPLOYMENT
GITOPS PADA KLASER KUBERNETES: STUDI KASUS INVENIORDM**

***IMPLEMENTATION OF NASI CUSTOM (NESTED ARGOCD SERVICE
INTEGRATION – CASCADING UNIFIED SERVICE TEMPLATE
ORCHESTRATION & MANIFEST) FOR GITOPS DEPLOYMENT
MANAGEMENT ON KUBERNETES CLUSTERS: A CASE STUDY OF
INVENIORDM***



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

PROPOSAL TUGAS AKHIR

**IMPLEMENTASI NASI CUSTOM (NESTED ARGOCD SERVICE
INTEGRATION – CASCADING UNIFIED SERVICE TEMPLATE
ORCHESTRATION & MANIFEST) UNTUK MANAJEMEN DEPLOYMENT
GITOPS PADA KLASER KUBERNETES: STUDI KASUS INVENIORDM**

***IMPLEMENTATION OF NASI CUSTOM (NESTED ARGOCD SERVICE
INTEGRATION – CASCADING UNIFIED SERVICE TEMPLATE
ORCHESTRATION & MANIFEST) FOR GITOPS DEPLOYMENT
MANAGEMENT ON KUBERNETES CLUSTERS: A CASE STUDY OF
INVENIORDM***

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

IMPLEMENTASI NASI CUSTOM (NESTED ARGOCD SERVICE INTEGRATION – CASCADING UNIFIED SERVICE TEMPLATE ORCHESTRATION & MANIFEST) UNTUK MANAJEMEN DEPLOYMENT GITOPS PADA KLASER KUBERNETES: STUDI KASUS INVENIORDM

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 9 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs.
Pembimbing

Nama Dosen Penguji 1
Ketua Penguji

Nama Dosen Penguji 2
Anggota Penguji

DAFTAR ISI

Halaman Judul	ii
Halaman Pengesahan	iii
INTISARI	vi
ABSTRACT	vii
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	4
1.6 Sistematika Penulisan	4
II PENELITIAN TERKAIT	6
2.1 Kubernetes dan Orkestrasi Kontainer	6
2.1.1 Manajemen Konfigurasi dan Configuration Drift	6
2.2 GitOps sebagai Paradigma Deployment Modern	7
2.3 ArgoCD dan Pola App of Apps	7
2.3.1 Pola App of Apps	8
2.3.2 Sync Waves dan Dependency Ordering	8
2.3.3 Self-Heal dan Drift Correction	8
2.3.4 AppProject dan Batas Keamanan	8
2.4 Pendekatan Deployment Alternatif	9
2.4.1 Helm Umbrella Chart	9
2.4.2 Kustomize Overlays	9
2.4.3 Deployment Manual/Click-ops	9
2.5 InvenioRDM sebagai Studi Kasus	10
2.6 Penelitian Terdahulu	10
III METODE DAN RANCANGAN PENELITIAN	12
3.1 Deskripsi Umum Penelitian	12

3.2	Metodologi	12
3.3	Perancangan Sistem	13
3.3.1	Arsitektur NASI CUSTOM	13
3.3.2	Sync Wave dan Urutan Deployment	13
3.3.3	Wiring Dependensi melalui ExternalName Service	14
3.4	Lingkungan Pengembangan	14
3.5	Metode Pengujian	15
3.5.1	E1: First-Attempt Success Rate	15
3.5.2	E2: Drift Detection & Recovery Time	16
3.5.3	E3: Blast Radius / Isolation	16
3.5.4	Baseline Perbandingan	17
IV	JADWAL PENELITIAN	18
4.1	Penjelasan Kegiatan	18

INTISARI

Implementasi NASI CUSTOM (Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest) untuk Manajemen Deployment GitOps pada Kluster Kubernetes: Studi Kasus InvenioRDM

Oleh

Muhammad Argya Vityasy

23/522547/PA/22475

Deployment aplikasi multi-service pada kluster Kubernetes tanpa pola yang terstruktur menghasilkan kekacauan operasional: layanan di-deploy tanpa urutan dependensi yang benar, perubahan konfigurasi tidak terdeteksi secara otomatis, dan satu kesalahan dapat merambat ke seluruh kluster. Penelitian ini mengimplementasikan dan mengevaluasi NASI CUSTOM (Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest), sebuah pola ArgoCD App of Apps dengan sync waves untuk manajemen deployment GitOps pada kluster Kubernetes, menggunakan InvenioRDM sebagai studi kasus. NASI CUSTOM mengorganisasi 16+ ArgoCD Application dalam hierarki induk-anak dengan anotasi sync wave yang menjamin urutan deployment dari batas keamanan hingga layanan aplikasi. Penelitian menggunakan metodologi Design Science Research dengan mengukur tiga metrik: (1) first-attempt success rate, yakni persentase deployment yang berhasil tanpa intervensi manual; (2) drift detection & recovery time, yakni waktu dari terjadinya penyimpangan konfigurasi hingga ArgoCD mendeteksi dan memulihkan secara otomatis; dan (3) blast radius, yakni jumlah aplikasi yang terpengaruh ketika konfigurasi satu aplikasi berubah. Hasil evaluasi dibandingkan dengan dua baseline: deployment manual/click-ops dan ArgoCD flat tanpa hierarki maupun sync waves. Penelitian ini diharapkan memberikan evaluasi kuantitatif pertama terhadap pola App of Apps pada aspek keandalan deployment, pemulihan drift, dan isolasi perubahan.

ABSTRACT

Implementation of NASI CUSTOM (Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest) for GitOps Deployment Management on Kubernetes Clusters: A Case Study of InvenioRDM

By

Muhammad Argya Vityasy

23/522547/PA/22475

Deploying multi-service applications on Kubernetes clusters without a structured pattern leads to operational chaos: services are deployed without correct dependency ordering, configuration changes go undetected, and a single mistake can cascade across the entire cluster. This research implements and evaluates NASI CUSTOM (Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest), an ArgoCD App of Apps pattern with sync waves for GitOps deployment management on Kubernetes clusters, using InvenioRDM as a case study. NASI CUSTOM organizes 16+ ArgoCD Applications in a parent-child hierarchy with sync wave annotations that guarantee deployment ordering from security boundaries to application services. The research employs Design Science Research methodology, measuring three metrics: (1) first-attempt success rate, the percentage of deployments that succeed without manual intervention; (2) drift detection & recovery time, the time from configuration drift occurrence to ArgoCD detection and auto-healing; and (3) blast radius, the number of applications affected when one application's configuration changes. Evaluation results are compared against two baselines: manual/click-ops deployment and flat ArgoCD without hierarchy or sync waves. This research is expected to provide the first quantitative evaluation of the App of Apps pattern in terms of deployment reliability, drift recovery, and change isolation.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform orkestrasi kontainer yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta developer secara global [Cloud Native Computing Foundation, 2024]. Kemampuan Kubernetes dalam mengelola deployment, scaling, dan self-healing menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan availability tinggi [Burns et al., 2016]. Namun, seiring bertambahnya jumlah layanan yang di-deploy pada sebuah kluster, kompleksitas operasional meningkat secara drastis. Sebuah aplikasi seperti InvenioRDM — platform repositori data penelitian open-source yang dikembangkan oleh CERN — terdiri dari setidaknya 16 komponen yang saling bergantung: layanan web, worker asinkron, database relasional, mesin pencari, cache, penyimpanan objek, ingress controller, manajer sertifikat, enkripsi secret, tunnel jaringan, kebijakan keamanan, sistem pemantauan, dan pencadangan [CERN, 2024].

Pengelolaan deployment untuk aplikasi multi-service semacam ini tanpa pola yang terstruktur menghasilkan kekacauan operasional (*deployment chaos*). Dalam pendekatan manual, operator menjalankan puluhan perintah `kubectl apply` atau membuat aplikasi satu per satu melalui antarmuka ArgoCD (*click-ops*). Akibatnya, tiga masalah muncul secara berulang. Pertama, *urutan dependensi tidak terjamin*: layanan aplikasi dapat di-deploy sebelum database atau cache siap, menyebabkan crash loop dan kegagalan deployment. Kedua, *configuration drift tidak terdeteksi*: perubahan manual langsung pada kluster (di luar Git) tidak ditemukan sampai menyebabkan insiden produksi. Ketiga, *blast radius tidak terkontrol*: satu perubahan konfigurasi pada satu layanan dapat memicu re-deploy pada layanan lain yang tidak terkait, atau sebaliknya, perubahan yang seharusnya merambat tidak terpropagasi [Zhang et al., 2026, Rahman et al., 2023].

GitOps menawarkan disiplin untuk mengatasi kekacauan ini. Dengan menjadikan repositori Git sebagai *single source of truth*, seluruh state infrastruktur tercatat, terverifikasi, dan dapat diaudit [CNCF GitOps Working Group, 2021]. ArgoCD mengimplementasikan prinsip-prinsip GitOps pada ekosistem Kubernetes: agen yang berjalan di dalam kluster secara periodik membandingkan state yang diinginkan di

Git dengan state aktual di kluster, dan secara otomatis menyelaraskan keduanya [Argo Project, 2024]. Namun, ArgoCD dalam konfigurasi datar (*flat*) — di mana setiap Application didefinisikan secara independen tanpa hierarki — tidak menyelesaikan persoalan urutan dependensi maupun isolasi perubahan. Enam belas Application tanpa sync wave saling berlomba untuk di-deploy pertama, dan perubahan pada satu direktori Git dapat memicu re-sync pada Application yang tidak terkait.

Pola *App of Apps* pada ArgoCD mengatasi keterbatasan ini dengan memperkenalkan hierarki induk-anak dan mekanisme *sync wave*. Sebuah Application induk mereferensikan direktori yang berisi manifest Application anak; ArgoCD secara otomatis membuat dan mengelola setiap anak. Anotasi `argocd.argoproj.io/sync-wave` pada setiap anak menentukan urutan deployment yang ketat, sehingga batas keamanan (*security policies*) di-deploy sebelum ingress controller, ingress controller sebelum manajer sertifikat, dan seterusnya hingga layanan aplikasi [Argo Project, 2024, Yuen et al., 2021]. Fitur `selfHeal` pada setiap Application memastikan bahwa setiap penyimpangan konfigurasi (*configuration drift*) dideteksi dan dikoreksi secara otomatis.

Meskipun pola App of Apps telah diadopsi secara luas di industri, evaluasi kuantitatif terhadap pola ini belum banyak dilakukan dalam literatur akademis. Pertanyaan mendasar yang belum dijawab: sejauh mana pola ini benar-benar meningkatkan keandalan deployment, mempercepat pemulihan dari drift, dan mengisolasi blast radius dibandingkan pendekatan tanpa hierarki? Penelitian mengenai drift dan misconfiguration pada Kubernetes menunjukkan bahwa 71% defect konfigurasi menyebabkan downtime atau degradasi layanan [Zhang et al., 2026], namun belum ada studi yang mengukur bagaimana pola GitOps hierarkis dapat mencegah atau memitigasi defect tersebut.

Berdasarkan kesenjangan tersebut, penelitian ini mengimplementasikan dan mengevaluasi **NASI CUSTOM** (*Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest*), sebuah realisasi konkret dari pola App of Apps untuk deployment multi-service InvenioRDM pada kluster Kubernetes. NASI CUSTOM mengorganisasi 16+ ArgoCD Application dalam hierarki induk-anak dengan 8 tingkat sync wave, dimulai dari security policies (wave –5) hingga dependensi database (wave 8). Evaluasi dilakukan dengan mengukur tiga metrik — *first-attempt success rate*, *drift detection & recovery time*, dan *blast radius* — dan membandingkannya dengan dua baseline: deployment manual/click-ops dan ArgoCD flat tanpa hierarki maupun sync waves.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana pola ArgoCD App of Apps dengan sync waves dapat menjamin urutan deployment yang benar pada aplikasi multi-service di klaster Kubernetes?
2. Sejauh mana pola NASI CUSTOM meningkatkan *first-attempt success rate* deployment dibandingkan pendekatan manual dan ArgoCD tanpa hierarki?
3. Bagaimana pola NASI CUSTOM mendeteksi dan memulihkan *configuration drift* secara otomatis, dan berapa lama waktu pemulihannya?
4. Bagaimana pola NASI CUSTOM membatasi *blast radius* ketika terjadi perubahan pada satu layanan — apakah perubahan tersebut terisolasi atau merambat ke layanan lain?

1.3 Batasan Masalah

Agar penelitian ini tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. Penelitian menguji pola App of Apps pada satu studi kasus: deployment InvenioRDM beserta 16 komponen infrastruktur dan dependensinya pada klaster Kubernetes.
2. Perbandingan dilakukan terhadap dua baseline: (a) deployment manual/click-ops, dan (b) ArgoCD flat Application tanpa hierarki dan tanpa sync waves.
3. Pengukuran dilakukan pada klaster Kubernetes tunggal (*Rancher-managed*); skenario multi-cluster tidak termasuk dalam ruang lingkup.
4. Komponen InvenioRDM sebagai aplikasi (fitur, antarmuka, logika bisnis) bukan variabel penelitian — InvenioRDM merupakan subjek uji, bukan objek penelitian.
5. Metrik yang diukur terbatas pada: *first-attempt success rate*, *drift detection & recovery time*, dan *blast radius/isolation*.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengimplementasikan pola ArgoCD App of Apps dengan sync waves (NASI CUSTOM) untuk deployment multi-service InvenioRDM pada kluster Kubernetes.
2. Mengukur *first-attempt success rate* deployment NASI CUSTOM dibandingkan baseline manual dan ArgoCD flat.
3. Mengukur *drift detection time* dan *recovery time* pada NASI CUSTOM dibandingkan ArgoCD flat.
4. Mengukur *blast radius* perubahan konfigurasi pada NASI CUSTOM — apakah perubahan pada satu Application mempengaruhi Application lain.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat berikut:

1. **Teoritis:** Memberikan evaluasi kuantitatif pertama terhadap pola ArgoCD App of Apps pada aspek keandalan deployment, pemulihan drift, dan isolasi perubahan — aspek yang belum ditemukan dalam literatur akademis eksisting.
2. **Praktis:** Menyediakan pola deployment GitOps yang terukur dan dapat diadopsi oleh organisasi yang mengelola aplikasi multi-service pada Kubernetes, mengurangi kekacauan operasional dan meningkatkan reliabilitas deployment.

1.6 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- **Bab I:** Pendahuluan — berisi latar belakang, rumusan masalah, batasan masalah, tujuan, manfaat, dan sistematika penulisan.
- **Bab II:** Penelitian Terkait — membahas Kubernetes, GitOps, ArgoCD dan pola App of Apps, pendekatan deployment alternatif, InvenioRDM sebagai studi kasus, serta penelitian terdahulu yang relevan.

- **Bab III:** Metode dan Rancangan Penelitian — menjelaskan metodologi Design Science Research, perancangan arsitektur NASI CUSTOM, tiga eksperimen pengukuran, bagan alir, lingkungan pengembangan, dan metode pengujian.
- **Bab IV:** Jadwal Penelitian — memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 8 bulan.

BAB II

PENELITIAN TERKAIT

2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah sistem orkestrasi kontainer open-source yang awalnya dikembangkan oleh Google berdasarkan pengalaman sistem Borg dan Omega, dan saat ini dikelola oleh Cloud Native Computing Foundation (CNCF) [Burns et al., 2016]. Kubernetes menyediakan abstraksi untuk automasi deployment, scaling, dan operasi aplikasi kontainerisasi [The Kubernetes Authors, 2024a]. Abstraksi utama dalam Kubernetes meliputi: *Pod* sebagai unit deploy terkecil, *Deployment* untuk manajemen replika aplikasi stateless, *StatefulSet* untuk aplikasi stateful, *Service* untuk eksposur jaringan, *ConfigMap* dan *Secret* untuk manajemen konfigurasi, serta *Ingress* untuk routing HTTP/HTTPS.

Dalam konteks deployment, Kubernetes menyediakan mekanisme rolling update yang secara default mengganti Pod lama dengan Pod baru secara bertahap. Namun, Kubernetes sendiri tidak menyediakan mekanisme bawaan untuk mengatur urutan deployment antar komponen yang saling bergantung. Sebuah aplikasi web yang memerlukan database tidak dapat secara deklaratif menyatakan “deploy setelah database siap” tanpa mekanisme eksternal. Keterbatasan ini menjadi masalah pada aplikasi multi-service di mana urutan dependensi bersifat kritis [Limoncelli et al., 2018].

2.1.1 Manajemen Konfigurasi dan Configuration Drift

Penelitian terbaru oleh Zhang et al. [2026] melakukan studi empiris terhadap 719 defect konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori defect, di mana 71% di antaranya menyebabkan downtime atau degradasi layanan. Studi ini menegaskan bahwa misconfiguration merupakan risiko operasional utama pada Kubernetes. Rahman et al. [2023] menambahkan bahwa security misconfiguration pada manifest Kubernetes open-source mencakup 11 kategori yang berulang, termasuk container yang berjalan sebagai root, tanpa drop capabilities, dan tanpa resource limits.

Configuration drift — situasi di mana state aktual kluster menyimpang dari state yang didefinisikan — dapat terjadi ketika operator melakukan perubahan langsung pada kluster melalui `kubectl edit` atau manual patch, melewati proses Git.

Deteksi dan koreksi drift secara manual memerlukan waktu dan rentan terhadap kelalaian [Pohjola, 2025]. GitOps dengan mekanisme self-healing menawarkan solusi otomatis untuk masalah ini.

2.2 GitOps sebagai Paradigma Deployment Modern

GitOps adalah paradigma untuk mengelola infrastruktur dan aplikasi yang menggunakan repositori Git sebagai sumber kebenaran tunggal (*single source of truth*). CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) state sistem yang diinginkan tersimpan dan berversi dalam Git; (3) perubahan state ditarik (*pulled*) secara otomatis oleh agen software; dan (4) agen software secara kontinu mendeteksi dan mengoreksi deviasi dari state yang diinginkan [CNCF GitOps Working Group, 2021].

Keunggulan GitOps dibandingkan deployment imperatif tradisional meliputi: audit trail lengkap melalui Git history, penerapan code review pada perubahan infrastruktur, rollback yang sederhana melalui `git revert`, dan reproduktibilitas konfigurasi di berbagai lingkungan [Yuen et al., 2021]. Dalam model *pull-based* GitOps, agen yang berjalan di dalam kluster secara periodik membandingkan state Git dengan state kluster dan menyelaraskan keduanya, menghilangkan kebutuhan akan akses kredensial kluster dari luar [Utami and Fatta, 2021].

Shrestha and Ali [2024] melakukan evaluasi empiris pendekatan GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, dan menemukan bahwa pendekatan GitOps memiliki drift detection time yang lebih cepat dibandingkan pendekatan imperatif. Kurrewar et al. [2025] mengkonfirmasi bahwa GitOps dengan ArgoCD menghasilkan deployment yang lebih andal untuk aplikasi pada Kubernetes.

2.3 ArgoCD dan Pola App of Apps

ArgoCD adalah tools GitOps yang dirancang khusus untuk ekosistem Kubernetes. ArgoCD mengimplementasikan model pull-based GitOps di mana agen yang berjalan di dalam kluster memantau repositori Git dan menyelaraskan state kluster dengan state yang didefinisikan [Argo Project, 2024]. ArgoCD mendukung berbagai format manifest termasuk plain YAML, Kustomize, Helm chart, dan Jsonnet. Setiap aplikasi didefinisikan sebagai *Application Custom Resource Definition* (CRD) yang mereferensikan sumber manifest di Git dan destination di kluster.

2.3.1 Pola App of Apps

Apps of Apps adalah pola arsitektur di mana sebuah Application ArgoCD induk (*parent*) mereferensikan direktori Git yang berisi manifest Application anak (*child*). Ketika *parent* disinkronkan, ArgoCD membaca manifest anak dan secara otomatis membuat serta mengelola setiap *child* Application [Argo Project, 2024, Yuen et al., 2021]. Pola ini menghasilkan hierarki yang memungkinkan: (1) *single entry point*, operator hanya mengelola satu Application induk untuk mengontrol seluruh stack; (2) organisasi modular, Application anak dikelompokkan berdasarkan domain atau tanggung jawab; dan (3) *reusability*, struktur dapat digunakan sebagai template untuk lingkungan berbeda [Costea et al., 2022].

2.3.2 Sync Waves dan Dependency Ordering

Sync wave adalah mekanisme ArgoCD untuk mengatur urutan deployment. Setiap Application dapat diberi anotasi `argocd.argoproj.io/sync-wave` dengan nilai numerik. ArgoCD menjamin bahwa semua Application pada wave n telah selesai dan sehat sebelum Application pada wave $n + 1$ dimulai [Argo Project, 2024]. Mekanisme ini menyelesaikan masalah dependensi deployment yang tidak ditangani oleh Kubernetes sendiri: batas keamanan dapat di-deploy pada wave -5 , ingress controller pada wave -4 , manajer sertifikat pada wave -2 , dan layanan aplikasi pada wave yang lebih tinggi.

2.3.3 Self-Heal dan Drift Correction

Ketika `syncPolicy.automated.selfHeal` diaktifkan, ArgoCD secara kontinu membandingkan state aktual kluster dengan state yang diinginkan di Git. Jika terjadi deviasi — misalnya, seorang operator mengubah ConfigMap langsung melalui `kubectl` — ArgoCD mendeteksi penyimpangan ini dan secara otomatis mengembalikan konfigurasi ke state yang didefinisikan di Git. Mekanisme ini merupakan implementasi langsung dari prinsip keempat OpenGitOps: *continuously reconciled* [CNCF GitOps Working Group, 2021].

2.3.4 AppProject dan Batas Keamanan

ArgoCD menyediakan *AppProject* CRD untuk mengendalikan akses dan batas deployment. *AppProject* mendefinisikan repositori sumber yang diizinkan, namespa-

ce destinasi, dan jenis resource klaster yang dapat dibuat oleh Application dalam project tersebut [Argo Project, 2024]. Dalam konteks pola App of Apps, AppProject berfungsi sebagai batas keamanan yang memastikan setiap Application hanya dapat mengakses sumber dan destinasi yang diizinkan, mencegah privilege escalation antar tim atau layanan.

2.4 Pendekatan Deployment Alternatif

Beberapa pendekatan alternatif untuk deployment aplikasi multi-service pada Kubernetes layak dibandingkan dengan pola App of Apps.

2.4.1 Helm Umbrella Chart

Helm mendukung *umbrella chart* yang menginstal beberapa sub-chart sekaligus [Helm Project, 2024]. Pendekatan ini mengemas seluruh stack dalam satu chart tunggal, namun memiliki beberapa keterbatasan: (1) tidak ada mekanisme bawaan untuk mengatur urutan antar sub-chart pada level runtime; (2) perubahan pada satu sub-chart mengharuskan re-deploy seluruh umbrella chart; dan (3) drift detection bergantung pada `helm diff` yang tidak kontinu [Yuen et al., 2021].

2.4.2 Kustomize Overlays

Kustomize menyediakan mekanisme overlay untuk mengkomposisi dan menyesuaikan manifest Kubernetes secara deklaratif [The Kubernetes Authors, 2024b]. Kustomize dapat digunakan bersama ArgoCD sebagai sumber manifest, namun sendirian tidak menyediakan mekanisme deployment ordering, drift detection, atau self-healing.

2.4.3 Deployment Manual/Click-ops

Pendekatan manual — baik melalui `kubectl apply` atau pembuatan Application secara manual melalui ArgoCD UI — merupakan baseline paling dasar. Nataraja [2025] membandingkan pendekatan deklaratif (GitOps/ArgoCD) dengan imperatif (Jenkins/kubectl) dan menemukan bahwa pendekatan deklaratif menghasilkan drift correction yang lebih cepat, compliance rate yang lebih tinggi, dan paparan kerentanan yang lebih rendah. Pendekatan manual tidak memiliki versioning otomatis, drift detection, atau rollback yang terstandarisasi.

2.5 InvenioRDM sebagai Studi Kasus

InvenioRDM adalah platform repositori data penelitian open-source yang dikembangkan oleh CERN dan komunitas internasional, dirancang untuk memenuhi prinsip FAIR (*Findable, Accessible, Interoperable, Reusable*) [CERN, 2024]. Arsitektur InvenioRDM bersifat microservices yang terdiri dari beberapa komponen: (1) *web application* berbasis Flask/Gunicorn, (2) *Celery workers* untuk pemrosesan asinkron, (3) PostgreSQL untuk penyimpanan data, (4) OpenSearch untuk indexing dan pencarian, (5) Redis sebagai cache dan message broker, serta (6) MinIO/S3 untuk penyimpanan objek [Przerwa and Rohr, 2025].

InvenioRDM dipilih sebagai studi kasus karena memenuhi tiga kriteria: (a) kompleksitas dependensi yang memerlukan ordering — web bergantung pada database, cache, dan search engine; (b) keberadaan komponen infrastruktur yang luas — mencakup ingress, TLS, secrets, monitoring, backup, dan kebijakan keamanan; dan (c) ketersediaan sebagai platform open-source yang dapat direproduksi [Chelakis, 2022, Fernández et al., 2016].

Dalam implementasi NASI CUSTOM, dependensi InvenioRDM diwiring melalui *ExternalName Service*: `invenio-postgresql` merujuk ke `postgres-rw.database.svc.cluster.local`, `invenio-redis` ke `redis-master.redis.svc.cluster.local`, dan `invenio-search` ke `opensearch-cluster-master.search.svc.cluster.local`. Pola ini memungkinkan InvenioRDM mengakses dependensi melalui nama layanan internal tanpa mengetahui lokasi fisik komponen, mendukung modularitas dan penggantian implementasi dependensi tanpa mengubah konfigurasi aplikasi.

2.6 Penelitian Terdahulu

Beberapa penelitian terkait telah dilakukan dalam konteks GitOps, ArgoCD, dan deployment Kubernetes. Utami and Fatta [2021] melakukan analisis perbandingan model deployment deklaratif pull-based (GitOps dengan ArgoCD) versus push-based pada Kubernetes, dan menemukan bahwa model pull-based lebih andal karena agen berjalan di dalam kluster dan tidak memerlukan kredensial eksternal. Shrestha and Ali [2024] mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, khususnya pada aspek drift detection dan remedy time.

Paavola [2021] membandingkan ArgoCD dan FluxCD untuk pengelolaan multi-application pada Kubernetes dan menyimpulkan bahwa ArgoCD lebih cocok un-

tuk skenario multi-application karena dukungan UI dan integrasi yang lebih baik. D'Amore [2021] mengimplementasikan ArgoCD untuk continuous deployment pada lingkungan hybrid cloud, namun tidak membahas pola App of Apps.

Matubber [2025] mengukur reconciliation time, scalability, dan reliability GitOps untuk infrastruktur K8s, termasuk self-healing terhadap configuration drift. Pohjola [2025] mempelajari mitigasi drift pada sistem Infrastructure-as-Code, memberikan kerangka konseptual untuk memahami sumber dan dampak drift.

Namun, dari seluruh penelitian di atas, belum ada yang secara khusus mengevaluasi pola ArgoCD App of Apps dengan metrik keandalan deployment (*first-attempt success rate*), pemulihan drift (*drift detection & recovery time*), dan isolasi perubahan (*blast radius*). Penelitian ini mengisi kesenjangan tersebut dengan memberikan evaluasi kuantitatif terhadap ketiga aspek tersebut.

BAB III

METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum Penelitian

Penelitian ini mengimplementasikan dan mengevaluasi NASI CUSTOM (*Nested ArgoCD Service Integration – Cascading Unified Service Template Orchestration & Manifest*), sebuah pola ArgoCD App of Apps dengan sync waves untuk manajemen deployment GitOps pada kluster Kubernetes. InvenioRDM digunakan sebagai studi kasus multi-service yang memerlukan ordering dependensi. Evaluasi dilakukan dengan mengukur tiga metrik — *first-attempt success rate*, *drift detection & recovery time*, dan *blast radius* — dan membandingkannya dengan dua baseline: deployment manual/click-ops dan ArgoCD flat tanpa hierarki maupun sync waves.

3.2 Metodologi

Metode penelitian yang digunakan adalah **Design Science Research (DSR)**. DSR dipilih karena penelitian ini membangun sebuah artefak (NASI CUSTOM) dan mengevaluasinya terhadap metrik yang terdefinisi untuk menghasilkan pengetahuan yang dapat digeneralisasi [Limoncelli et al., 2018]. Tahapan DSR dalam penelitian ini terdiri dari lima tahapan:

1. **Tahap 1: Studi Literatur** — Pengumpulan dan analisis literatur mengenai Kubernetes, GitOps, ArgoCD, pola App of Apps, configuration drift, serta arsitektur InvenioRDM.
2. **Tahap 2: Perancangan NASI CUSTOM** — Perancangan arsitektur App of Apps, struktur direktori repositori Git, definisi sync wave, AppProject, dan mekanisme wiring dependensi melalui ExternalName Service.
3. **Tahap 3: Implementasi** — Pembangunan repositori GitOps, konfigurasi 16+ ArgoCD Application, deployment InvenioRDM beserta seluruh komponen infrastruktur, serta konfigurasi baseline perbandingan (ArgoCD flat).
4. **Tahap 4: Pengujian Terukur** — Pelaksanaan tiga eksperimen (E1, E2, E3) yang masing-masing diulang untuk reliabilitas.

5. **Tahap 5: Analisis dan Evaluasi** — Interpretasi hasil eksperimen, perbandingan dengan baseline, dan pembahasan implikasi.

3.3 Perancangan Sistem

3.3.1 Arsitektur NASI CUSTOM

Arsitektur NASI CUSTOM terdiri dari empat lapisan yang saling berinteraksi:

- **Lapisan 1: Git Repository** — *Single source of truth* yang menyimpan seluruh manifest Kubernetes, ArgoCD Application, Helm values, dan konfigurasi SealedSecret dalam bentuk Infrastructure as Code. Repositori terstruktur dalam dua direktori utama: `argocd/` (definisi Application dan AppProject) dan `k8s/` (manifest infrastruktur dan aplikasi).
- **Lapisan 2: ArgoCD App of Apps** — Application induk (`argocd/root.yaml`) mereferensikan direktori `argocd/apps/` yang berisi 16+ Application anak. ArgoCD membaca direktori ini dan secara otomatis membuat serta mengelola setiap anak. Setiap anak memiliki anotasi `sync wave` yang menjamin urutan deployment.
- **Lapisan 3: Kubernetes Cluster** — Platform orkestrasi yang menjalankan seluruh komponen. AppProject `infra` mendefinisikan batas keamanan: repositori sumber yang diizinkan, namespace destinasi, dan resource yang dapat dibuat.
- **Lapisan 4: InvenioRDM dan Infrastruktur** — Kumpulan layanan yang terdiri dari komponen aplikasi (web, worker) dan komponen infrastruktur (PostgreSQL, OpenSearch, Redis, MinIO, Traefik, Cert-manager, Sealed Secrets, Cloudflare Tunnel, Velero, Monitoring, Security Policies).

3.3.2 Sync Wave dan Urutan Deployment

Tabel 3.1 menunjukkan urutan deployment dalam NASI CUSTOM berdasarkan anotasi `sync wave`. ArgoCD menjamin bahwa semua Application pada wave n telah sehat sebelum Application pada wave $n + 1$ dimulai.

Tabel 3.1: Urutan Sync Wave NASI CUSTOM

Wave	Application	Tujuan
−5	security-policies	Batas keamanan, NetworkPolicy, R
−4	traefik, cloudflared	Ingress controller + tunnel jaringan
−3	sealed-secrets, argocd-self	Enkripsi secret + manajemen Argo
−2	cert-manager	Manajemen sertifikat TLS
2	velero	Pencadangan klaster
5	monitoring	Prometheus + Grafana
7	invenio-bootstrap	Layanan InvenioRDM (web, work
8	invenio-opensearch, invenio-postgresql, invenio-redis	Dependensi database

3.3.3 Wiring Dependensi melalui ExternalName Service

InvenioRDM mengakses dependensi melalui ExternalName Service yang berfungsi sebagai abstraksi. Tabel 3.2 menunjukkan pemetaan layanan internal ke layanan fisik.

Tabel 3.2: Pemetaan ExternalName Service pada InvenioRDM

Layanan Internal	ExternalName	Namespace
invenio-postgresql	postgres-rw.database.svc.cluster.local	database
invenio-redis	redis-master.redis.svc.cluster.local	redis
invenio-search	opensearch-cluster-master.search.svc.cluster.local	search

Pola ini memungkinkan substitusi implementasi dependensi tanpa mengubah konfigurasi aplikasi InvenioRDM.

3.4 Lingkungan Pengembangan

Lingkungan pengembangan yang digunakan dalam penelitian ini adalah:

- **Sistem Operasi:** macOS (workstation), Ubuntu 22.04 LTS (node klaster).
- **Kubernetes:** Rancher-managed cluster, K8s v1.29+.
- **ArgoCD:** v2.12+.
- **GitOps Repository:** GitHub (`invenio-rdm-gitops`).

- **Package Manager:** Helm v3.14+, Kustomize.
- **Container Runtime:** containerd.
- **Secret Management:** Sealed Secrets v0.27+.
- **Ingress:** Traefik Helm chart.
- **Monitoring:** kube-prometheus-stack v69.6.0.
- **InvenioRDM:** v12.x (custom image dari GHCR).

3.5 Metode Pengujian

Pengujian dilakukan melalui tiga eksperimen yang masing-masing mengukur satu metrik utama. Setiap eksperimen diulang 5 kali untuk memastikan reliabilitas hasil.

3.5.1 E1: First-Attempt Success Rate

Tabel 3.3: Desain Eksperimen E1 — First-Attempt Success Rate

Parameter	Deskripsi
Metrik	Persentase deployment yang berhasil tanpa intervensi manual
Cara Ukur	Deploy dari awal (hapus semua Application, re-sync) 5 kali per pendekatan; hitung jumlah keberhasilan dibagi total percobaan
Pendekatan	(a) Manual/click-ops, (b) ArgoCD flat (tanpa hierarki, tanpa sync wave), (c) NASI CUSTOM
Kriteria Sukses	Seluruh Application menunjukkan status Healthy dan Synced; tidak diperlukan <code>argocd app sync</code> manual, <code>kubectl rollout restart</code> , atau perbaikan konfigurasi
Kriteria Gagal	Setiap intervensi manual setelah sync awal dianggap kegagalan

3.5.2 E2: Drift Detection & Recovery Time

Tabel 3.4: Desain Eksperimen E2 — Drift Detection & Recovery Time

Parameter	Deskripsi
Metrik	Waktu (detik) dari drift terjadi hingga ArgoCD mendeteksi + waktu dari deteksi hingga konfigurasi dipulihkan
Cara Ukur	Introduce drift secara deliberat (edit ConfigMap atau Deployment melalui <code>kubectl</code> langsung, bypass Git); catat timestamp t_0 (drift dibuat), t_1 (ArgoCD mendeteksi via <code>argocd app diff</code>), t_2 (ArgoCD selesai memulihkan via <code>selfHeal</code>)
Pendekatan	(a) ArgoCD flat (dengan <code>selfHeal</code>), (b) NASI CUSTOM (dengan <code>selfHeal</code>)
Jenis Drift	Perubahan isi ConfigMap, perubahan replika Deployment, perubahan image tag
Pengulangan	3 jenis drift $\times$ 5 pengulangan = 15 pengukuran per pendekatan

3.5.3 E3: Blast Radius / Isolation

Tabel 3.5: Desain Eksperimen E3 — Blast Radius / Isolation

Parameter	Deskripsi
Metrik	Jumlah Application yang melakukan re-sync ketika satu Application berubah
Cara Ukur	Ubah satu parameter pada Git (misal: image tag Invenio-RDM); amati berapa Application lain yang menunjukkan aktivitas <code>status.operationState</code> setelah perubahan
Pendekatan	(a) ArgoCD flat, (b) NASI CUSTOM
Hasil Diharapkan	NASI CUSTOM: hanya 1–2 Application re-sync (yang berubah + parent). ArgoCD flat: berpotensi lebih banyak karena <code>shared source path</code>
Pengulangan	5 perubahan berbeda pada 5 Application berbeda per pendekatan

3.5.4 Baseline Perbandingan

Untuk eksperimen E1, E2, dan E3, dua baseline digunakan:

1. **Manual/click-ops:** Deployment dilakukan melalui `kubectl apply` satu per satu atau pembuatan Application melalui ArgoCD UI tanpa automasi. Baseline ini hanya relevan untuk E1 karena tidak memiliki mekanisme drift detection atau blast radius yang terukur.
2. **ArgoCD flat:** 16+ Application didefinisikan secara independen tanpa parent Application, tanpa hierarki, dan tanpa anotasi sync wave. Semua Application disimpan dalam satu direktori yang sama. Baseline ini mengisolasi pengaruh hierarki dan ordering terhadap metrik yang diukur.

BAB IV

JADWAL PENELITIAN

Penelitian ini direncanakan untuk dilaksanakan selama delapan bulan, dari bulan Januari 2026 hingga Agustus 2026. Jadwal penelitian disusun berdasarkan tahapan Design Science Research (DSR) yang telah dijelaskan pada Bab III.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol • menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan							
	Jan	Feb	Mar	Apr	Mei	Jun	Jul	Agu
Studi Literatur	•	•						
Perancangan NASI CUSTOM		•	•					
Implementasi			•	•	•			
Pengujian (E1, E2, E3)					•	•		
Analisis & Evaluasi						•	•	
Penulisan Laporan	•	•	•	•	•	•	•	•
Seminar / Sidang								•

4.1 Penjelasan Kegiatan

1. Studi Literatur (Januari – Februari 2026)

Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai Kubernetes, GitOps, ArgoCD, pola App of Apps, configuration drift, dan arsitektur InvenioRDM. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), dokumentasi resmi, buku teks, dan artikel teknis.

2. Perancangan NASI CUSTOM (Februari – Maret 2026)

Tahap ini meliputi perancangan arsitektur App of Apps, struktur direktori repositori Git, definisi 8 tingkat sync wave, desain AppProject dengan RBAC, desain wiring dependensi melalui ExternalName Service, serta desain skenario eksperimen E1, E2, dan E3.

3. Implementasi (Maret – Mei 2026)

Tahap implementasi meliputi: pembuatan repositori GitOps, konfigurasi ArgoCD, pembuatan Application induk dan 16+ Application anak dengan anotasi sync wave, konfigurasi InvenioRDM (web, worker, setup job, ExternalName Service), konfigurasi komponen infrastruktur (Sealed Secrets, Traefik, Cert-manager, Cloudflare Tunnel, Velero, MinIO, Monitoring, Security Policies), serta pembuatan baseline ArgoCD flat untuk perbandingan.

4. Pengujian E1, E2, E3 (Mei – Juni 2026)

Pelaksanaan tiga eksperimen sesuai desain pada Bab III. E1: first-attempt success rate (5 pengulangan per pendekatan). E2: drift detection & recovery time (3 jenis drift $\times$ 5 pengulangan per pendekatan). E3: blast radius (5 perubahan berbeda per pendekatan). Data dikumpulkan dalam bentuk timestamp dan status dari `argocd CLI` serta `kubectl`.

5. Analisis dan Evaluasi (Juni – Juli 2026)

Analisis hasil eksperimen mencakup: perbandingan first-attempt success rate antara tiga pendekatan, perbandingan drift detection time dan recovery time antara dua pendekatan, serta perbandingan blast radius (jumlah Application yang terpengaruh) antara dua pendekatan. Pembahasan mencakup interpretasi hasil, identifikasi faktor yang mempengaruhi, dan implikasi terhadap praktik GitOps.

6. Penulisan Laporan (Januari – Agustus 2026)

Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan semester, draft skripsi lengkap pada akhir penelitian.

7. Seminar / Sidang (Agustus 2026)

Presentasi dan sidang tugas akhir di depan dosen penguji.

DAFTAR PUSTAKA

- Argo Project. Argocd documentation, 2024. URL <https://argo-cd.readthedocs.io>.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.
- CERN. Inveniordm documentation, 2024. URL <https://inveniordm.docs.cern.ch>.
- K. Chelakis. Creating an open archival information system compliant archive for cern. Master’s thesis, CERN, 2022.
- Cloud Native Computing Foundation. Cnfc cloud native definition v1.0, 2024. URL <https://www.cncf.io/about/who-we-are>.
- CNCF GitOps Working Group. Opengitops principles v1.0.0. <https://opengitops.dev>, 2021.
- Liviu Costea, Stefan Economakis, and Alexander Matyushentsev. *Argo CD in Practice: The GitOps Way of Managing Cloud-Native Applications*. Packt Publishing, 2022.
- M. D’Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master’s thesis, Politecnico di Torino, 2021.
- J. D. Fernández, P. Fokianos, P. Herterich, and T. Šimko. Cern analysis preservation: A novel digital library service to enable reusable and reproducible research. In *Research and Advanced Technology for Digital Libraries*. Springer, 2016.
- Helm Project. Helm documentation, 2024. URL <https://helm.sh/docs>.
- S. Kurrewar, S. Dhomane, and A. Dahake. Streamlining kubernetes deployments through gitops methodologies. In *2025 IEEE International Conference on Electronics and Sustainable Communication Technologies*. IEEE, 2025.
- Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

- M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.
- P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.
- E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.
- O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.
- K. Przerwa and S. Rohr. A modern open source integrated library system by invenio. In *International Conference on Theory and Practice of Digital Libraries (TPDL)*. Springer, 2025.
- A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.
- R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.
- The Kubernetes Authors. Kubernetes documentation, 2024a. URL <https://kubernetes.io/docs>.
- The Kubernetes Authors. Kustomize documentation, 2024b. URL <https://kustomize.io>.
- Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.
- Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.
- Y. Zhang, U. Paul, M. d'Amorim, and A. Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 2026.